

RELATIONAL DATABASE LANGUAGE AND

COMPONENTS OF SQL → Structured Query Language

2023

What is SQL write its type

2M

DBMS Language / SQL commands

DATABASE LANGUAGES

SQL

SQL is a structured query language
is standard language
used to create, store,
retrieve, update,
manage data in
a database.
ex - select, insert

DBMS Language

DBMS Language

DDL

DCL

DML

TCL

SQL commands

1 type

10M

DDL

5M

Data Definition Language

CREATE

ALTER

DROP

RENAME

TRUNCATE

COMMENT

DML

Data Manipulation Language

SELECT

INSERT

UPDATE

DELETE

MERGE

CALL

EXPLAIN PLAN

LOCK TABLE

DCL

Data Control Language

GRANT

REVOKE

TCL

Transaction Control

COMMIT

ROLLBACK

SAVE POINT

SET TRANSACTION

Types

10M

Explain DDL, DML, DCL with suitable Syntax

DDL - Data Definition Language

We specify a database schema by a set of definitions
expressed by a special language called a data-
definition language (DDL)

4. Create

SQL → one specific language used to work with relational database

Database language

Database language are used to define,
manipulate, query & control data
in database

ex - DDL, DML, DCL, TCL
all language used in DBMS

Rajguru

1. SQL

Language commands

used to define & change the structure of database

1. Create Table

Used to create the table

Syntax CREATE TABLE TABLE NAME (COLUMN1 DATATYPE, COLUMN2 DATATYPE...);
create table student (regno. int, name varchar(10));

2. Drop Table

Used to delete the table

Syntax DROP TABLE TABLE-NAME;
drop table student;

3. Alter table

Explain ALTER (sub qn)

Used to alter the column/structure

Syntax ALTER TABLE TABLE-NAME ADD COLUMN-NAME data type;
alter table student add dob date;

↑ keyword

• ALTER TABLE TABLE-NAME DROP COLUMN COLUMN-NAME;
alter table student drop dob;

4. Rename

↑ keyword.

• Rename column

how do you rename table?
write syntax of it

Syntax ALTER TABLE table-name RENAME COLUMN old-name TO new-name;

• Rename Table table rename student to students;

Syntax RENAME table-name to new-table-name;

5. alter table student rename column dob to dateofbirth;

1. column

Explain DDL, DML, DCL, explain various DML operation with ex.

→ 27 Data - Manipulation Language (DML)

DDL is create and modify data in table.

Data manipulation is the retrieval of information stored in the database, the insertion of new information into the database, the deletion of information from the database.

DML commands

SM what is file? ✓✓✓
Explain various operations performed on a file

create, insert, retrieve, update, delete

✓ 1 Insert Table

Used to ~~extract~~ insert values in table.

Syntax INSERT INTO table-name VALUES (

value 1, value 2, value 3, ...);

Insert into student values (regno 101, name char(10));

✓✓✓ 2 Select explain select.

used to extract data from database

Syntax SELECT column-name FROM table-name;

Select * from student;

✓ 3. Update

Update the data within the table

Syntax UPDATE table-name SET column-name 1 = 'value 1'

column-name 2 = 'value 2' WHERE column-name =

'value 3'; update student set name = 'Ajay' where regno = 101;

✓ 4. Delete

Used to delete the rows in table

DELETE FROM table-name WHERE column-name = 'value';

Delete From student where regno = '101';

all 3
10M
It is used to retrieve the stored or saved data. DDL, DCL, DML - 10M

③ Data Control Language (DCL)

It is used to retrieve the stored or saved data.

① Grant: It is used to give user access privileges to a database. used to give permission. Allow users to access table.

② Revoke - It is used to take back permission from the user.

Syntax grant privilege-name on object-name to {
user-name / public / Role-name

grant permission
on table-name

Ex:- grant insert, select on Student-list to xyz
from wernam;

Syntax grant privilege-name on object-name to {
user-name / public / Role-name} ~~Revoke table privi~~
on table-name
to user-name

Ex: revoke insert, selection on Student-list to xyz

④ Transaction Control Language (Query Language) (TCL)

Transaction Control Language is used to save the changes made by the DML statement. TCL can be grouped into a logical transaction.

① commit

It is used to save the transaction on the database.

Syntax :- commit; (Save the changes made

in database)

② Roll back

It is used to restore the database to original since the last commit cancel the changes made by the commit and return database to its previous state.

SM

SM

✓ Explains Datatypes in SQL

Date

Data Types in SQL

1. Numeric data types - Numeric data types include integer numbers of various size (INTEGER or INT and SMALL INT)
Floating - point real numbers of various precision (FLOAT or REAL, and DOUBLE PRECISION)
2. Character
String data types are either fixed length CHAR(n) or character (n), where n is the number of characters. VARCHAR or CHAR is used to store character and alphanumeric character
• for example, if the value 'Rahul' is for an attribute of type CHAR(10)
3. Boolean
Data type has the traditional values of TRUE or FALSE
4. Date data type has 10 positions and its components are DAY, MONTH and YEAR in the form DD-MM-YYYY
• The time data type has at least eight positions with the components hour, minute and second in the form HH:MM:SS
• only valid dates and time should be allowed by the SQL implementation
• Time, Year, DateTime, Date, TimeStamp.

SET OPERATIONS

Operators in SQL

The SQL reserved words and characters are called Operators, which are used with a where clause in a SQL query to perform certain operations.

- | | | |
|------------------------------|-----------|-------------------|
| → 1. Arithmetic Operator | ① AND | ④ like |
| → 2. Comparison operator | ② OR | ⑤ Not like |
| → 3. Bitwise operator | ③ NOT | ⑥ is null |
| → 4. Logical Operator | ④ LN | ⑦ is not null |
| → 5. Assignment operator | ⑤ Not in | ⑧ exists |
| → 6. Relational operator | ⑥ between | |
| | ⑦ ALL | |
| | ⑧ Any | |
| → 1. <u>Logical Operator</u> | ⑨ total | → used to combine |

- ① AND operator
- Logical operator used to filter record combining multiple conditions.
- Used within where or having clause.

Syntax

Select column ... column N From tablename
where condition1 and condition2 and condition N;
Select * from student where regno = 101 and name = 'Arjun';

- ② OR operator
- Logical operator is used to filter record combining multiple conditions used within where or having clause.
- A record is included in result set if at least one condition is true.

Syntax

Select column1, column N From tablename
where condition1 OR condition2 OR condition N;
Select * from std where m1 = 52 or name = 'Arjun';

→ ③ Not operator

In SQL shows the record from the table if the condition evaluates to false. It is always used with the where clause.

select * from student where NOT marks = 80;

Syntax

select column1, column2 ..., columnN From tableN
where not condition;

→ ④ Between operator

In SQL shows the record within the range mentioned in the SQL query

select * from student where regno between 101 and 105;

Syntax

select column-name1, column-name2 ...,
column-name from table-name where column-name
between value1 and value2; or name 'yes' between
value1 and value2;

→ ⑤ In operator

In operator allows you to specify multiple values in a where clause

In operator is used to check whether a value matches any

Syntax

select column-name value in a
FROM table-name specified list of
WHERE column-name in (value1, value2);

select * from student where place in ('Kargil', 'Kumbh')

Between

select column-name1, column-name2 ..., column-nameN
from table-name where column-name between
value1 and value2;

FROM table-name

SELECT min (roll) from std;

MIN

LOGICAL OPERATORS (continuation) I & II

write the usage of NOT IN, like - 2M (2023)

→ ⑥ NOT IN → opposite of IN operator

The NOT IN operator used to compare a column with one or more than one value → used to select values
It always used with where clause not present in a list

Syntax SELECT column-name (S) FROM table-name
WHERE column-name NOT IN (value 1, value 2, ...);
select name from students where not in (18, 20);

→ ⑦ ALL operator → compares with all values

- ~~ALL~~ ALL operator is used to compare a value to a list of values or result set returned by a subquery.
- The ALL operator must be preceded by an comparison operator such as =, !=, >, >=, <, <=
- the list or subquery must be surrounded by the parentheses. Returns true only if all match

Syntax SELECT * FROM table-name WHERE column-name
> ALL (value 1, value 2, value 3, ...);

any small

select * from std where roll < all (41, 87, 90, 79);

ALL operator is used to compare a value with all values in another query (subquery)

comparing with a value with a list of values or not set returned by a subquery

→ ANY operator is used to compare a value to a list of values or not set returned by a subquery

The ANY operator must be preceded by a comparison operator, such as $=$, $<$, $>$, $<=$, $>=$, $<>$, etc. The list of subquery must be surrounded by the parenthesis. It returns true if any 1 value satisfies the condition

Syntax SELECT * FROM table-name WHERE column-name > ANY (value 1, value 2, value 3, ...);

SELECT * FROM stud where m1 > ANY (41.8, 99.9, 79.9)

→ LIKE operator LIKE operator is used to check whether values in a column match a specified pattern

used for matching any number of characters

used for matching exactly one character

used for matching string of characters

Syntax SELECT column-name FROM table-name WHERE column-name LIKE 'pattern';

SELECT column-name FROM table-name WHERE column-name LIKE '1';

SELECT * FROM beo4b where name like 'A%';

↑ 1 underscore - 1 row

name start with A

name consist of A in with any

NOT LIKE

used to find values not matching pattern

SELECT column-name FROM table-name WHERE column-name NOT LIKE 'pattern';

SELECT name FROM students where name NOT LIKE 'A%';

SELECT * FROM beo4b where name like 'A%';

non empty

→ IS NULL and IS NOT NULL

IS NULL and IS NOT NULL operators to check if a value in a column or an expression is NULL or not

NULL is missing information or the information is not applicable

both return boolean value (true/false)

Syntax for IS NULL → to check empty

SELECT column-name FROM table-name WHERE column-name IS NULL;

SELECT * FROM stud where m1 IS NULL;

SELECT column-name FROM table-name WHERE column-name IS NOT NULL;

SELECT * FROM stud where m1 IS NOT NULL;

→ EXISTS operator EXISTS operator is a Boolean operator that returns either true or false

returns true if data exists

Syntax SELECT column-name FROM table-name WHERE EXISTS (subquery);

SELECT * FROM beo4b where exists (SELECT name FROM beo4b);

SELECT * FROM students where exists (SELECT name FROM students);

LIKE operator used to find values that match specific pattern

SELECT * FROM students where name like 'A%';

SELECT name FROM students where name like 'A%';

SELECT * FROM beo4b where name like 'A%';

SELECT * FROM beo4b where name like 'A%';

SELECT * FROM beo4b where name like 'A%';

SELECT * FROM beo4b where name like 'A%';

SELECT * FROM beo4b where name like 'A%';

Clause

clause is used to retrieve the data from records present in database

① WHERE clause

The WHERE clause is used to filter records

• **SELECT** column1, column2, ... column n, **FROM** table-name **WHERE** condition; **FOR** ...

② ORDER BY

The ORDER BY keyword is used to sort the result set in ascending or descending order

• **SELECT** column1, column2, ... column n, **FROM** table-name **ORDER BY** column1, column2, ... column n **ASC** | **DESC**;

SELECT * **FROM** stu **ORDER BY** rollno **ASC** | **DESC**;

Imp 5th

Explain group by & having clause

GROUP BY → It displays all of the select is no repeated

• It is used with SQL SELECT statement, aggregation function and statement follows the WHERE clause (optional)

Syntax **SELECT** column-name1, column-name2, ... **FROM** table-name **GROUP BY** column-name;

Syntax **SELECT** aggregate function (column1, column2, ... column n) "expression" **FROM** table-name

SELECT name **FROM** student **GROUP BY** name;

WHERE condition **GROUP BY** column1, column2, ... column n;

④ HAVING

having clause is used to specify a search condition for a group or an aggregate.

Syntax

SELECT column-name "expression" **FROM** table-name **GROUP BY** column-name;

HAVING condition;

⑤ DISTINCT

To show distinct (different) values from column eliminate duplicate rows & display a unique list of values

Syntax

SELECT **DISTINCT** column-name **FROM** table-name

SELECT **DISTINCT** name **FROM** student;

SELECT **DISTINCT** name **FROM** student;

(name1, name2) X AND Y

(name1, name2) X AND Y

SQL FUNCTIONS

SQL updated Explain Aggregate Function in SQL

SQL Aggregate Function

→ 1) COUNT Function
Count function is used to count the number of rows in a database table.

Syntax: `SELECT COUNT (column-name) FROM table-name WHERE condition;`

SELECT COUNT (column-name) FROM table-name WHERE condition;

2) SUM Function

Sum function is used to calculate the sum of all selected columns.

Syntax: `SELECT SUM (column-name) "expression" FROM table-name WHERE condition;`

SELECT SUM (column-name) FROM table-name WHERE condition;

3) AVG Function

The AVG function is used to calculate the average value of the numeric type.

Syntax

SELECT AVG (column-name) FROM table-name;

SELECT AVG (column-name) FROM table-name;

4) MAX Function

MAX function is used to find the maximum value of a certain column.

Syntax

SELECT MAX (column-name) "expression"

FROM table-name;

SELECT MAX (column-name) FROM table-name;

Aggregate function are functions that perform calculation on multiple rows & return a single value.

5) MIN Function

MIN function is used to find the minimum value of a certain column.

Syntax: `SELECT MIN (column-name) "expression" FROM table-name;`

SELECT MIN (column-name) FROM table-name;

LOGICAL OPERATORS

→ 6) NOT IN
The NOT IN operator is used to compare a column with one or more than one value.

Syntax: `SELECT column-name (S) FROM table-name WHERE column-name NOT IN (value 1, value 2, ...);`

SELECT column-name FROM table-name WHERE column-name NOT IN (value 1, value 2, ...);

7) ALL Operator

ALL operator is used to compare a value to a list of values or value set returned by a subquery.

Syntax: `SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);`

SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);

Syntax: `SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);`

SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);

SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);

SELECT * FROM table-name WHERE column-name ALL (value 1, value 2, value 3, ...);

As Note - Used to check whether container is empty

any operator is used to return a subquery

The ANY operator must be followed by a list of operators such as $=, <, >, >=, <=$

comparison of subjects must be surrounded by the
the list of subjects must be surrounded by the

paraphrase. = The return to condition
satisfy the condition

SELECT * FROM table-name WHERE column-name

select 1 from sid where m1 = 0 and m2 = 1

→ ~~Q~~ Like operator not like
to test whether value in a column

match a specified pattern

- used for matching any number of character

→ costo for maintaining a world class string of food products for

column name	from table name	where
select column name	from table name	where

room for the room when

```
select * from table-name  
column-name like ' % ';
```

↑ 1 underscore - 1 P108410

select # from bread where name like ('Q.%')

nom slot with a

consist of a in vitro display

NOT LIKE.

select column-name from table-name where

column-name not like 'y';

6A% 1.9

② ~~It is null~~ and ~~It is not null~~

- IS NULL and IS NOT NULL operators to check if

a value in a column or an expression is NULL or

- Null is missing information or the information

is not applicable • both identity criterion and total

Syntax for IS NULL → to check null

select column-name FROM table-name WHERE

id	name	age	sex	status
1	John	20	M	single
2	Jane	22	F	single
3	John	25	M	married
4	Jane	28	F	married
5	John	30	M	married
6	Jane	32	F	married
7	John	35	M	married
8	Jane	38	F	married
9	John	40	M	married
10	Jane	42	F	married
11	John	45	M	married
12	Jane	48	F	married
13	John	50	M	married
14	Jane	52	F	married
15	John	55	M	married
16	Jane	58	F	married
17	John	60	M	married
18	Jane	62	F	married
19	John	65	M	married
20	Jane	68	F	married
21	John	70	M	married
22	Jane	72	F	married
23	John	75	M	married
24	Jane	78	F	married
25	John	80	M	married
26	Jane	82	F	married
27	John	85	M	married
28	Jane	88	F	married
29	John	90	M	married
30	Jane	92	F	married
31	John	95	M	married
32	Jane	98	F	married
33	John	100	M	married
34	Jane	102	F	married
35	John	105	M	married
36	Jane	108	F	married
37	John	110	M	married
38	Jane	112	F	married
39	John	115	M	married
40	Jane	118	F	married
41	John	120	M	married
42	Jane	122	F	married
43	John	125	M	married
44	Jane	128	F	married
45	John	130	M	married
46	Jane	132	F	married
47	John	135	M	married
48	Jane	138	F	married
49	John	140	M	married
50	Jane	142	F	married
51	John	145	M	married
52	Jane	148	F	married
53	John	150	M	married
54	Jane	152	F	married
55	John	155	M	married
56	Jane	158	F	married
57	John	160	M	married
58	Jane	162	F	married
59	John	165	M	married
60	Jane	168	F	married
61	John	170	M	married
62	Jane	172	F	married
63	John	175	M	married
64	Jane	178	F	married
65	John	180	M	married
66	Jane	182	F	married
67	John	185	M	married
68	Jane	188	F	married
69	John	190	M	married
70	Jane	192	F	married
71	John	195	M	married
72	Jane	198	F	married
73	John	200	M	married
74	Jane	202	F	married
75	John	205	M	married
76	Jane	208	F	married
77	John	210	M	married
78	Jane	212	F	married
79	John	215	M	married
80	Jane	218	F	married
81	John	220	M	married
82	Jane	222	F	married
83	John	225	M	married
84	Jane	228	F	married
85	John	230	M	married
86	Jane	232	F	married
87	John	235	M	married
88	Jane	238	F	married
89	John	240	M	married
90	Jane	242	F	married
91	John	245	M	married
92	Jane	248	F	married
93	John	250	M	

SYNLOG	NOT NULL	FROM table-name where

select column_name is not null;

if exists operator NOT EXISTS

exists operator $\exists$ is a Boolean operator that returns

either true or false. A returning time t_1 does exist.

Syntax

SELECT column-name FROM table-name WHERE
EXISTS (subquery);

Select * 10m bread where exists (Select name first)

bca4b),

Select * from student where wish (select name from student);

1. Explain how to find value that

match specific path in

select name from student where name like 'C%'

6-1-D

Rajgurun

Number Functions in SQL

1. **ABS()** → Absolute value.
 Removes the negative sign and returns the +ve value.
 ex: Select ABS (-25) FROM DUAL;

2. **CEIL()** → Ceiling Function
 Rounds the number up to the next integer.
 Even if the decimal is small, it moves to the next higher integer.
 ex: Select CEIL (4.2) FROM DUAL;

3. **FLOOR()** → Floor Function
 Rounds the number down to the lower integer.
 Removes decimal part and goes to lower integer.
 Always goes down.
 ex: Select FLOOR (4.8) FROM DUAL;

4. **ROUND()** - Round function
 Rounds a number to a specified number of decimal places.
 Used for accurate calculations.
 Example: Select Round (45.678, 2) FROM DUAL;

5. **MOD()** - Modulus Function
 Returns the remainder of a division.
 Example: Select MOD (10, 3) FROM DUAL;

6. **TRUNC()** - Truncate Function
 Truncates a number to a specified number of decimal places.
 Used for truncating data.
 Example: Select TRUNC (45.678, 2) FROM DUAL;

7. **SIGN()** - Sign Function
 Returns the sign of a number.
 1 for positive, -1 for negative, 0 for zero.
 Example: Select SIGN (10) FROM DUAL;

8. **POWER()** - Power Function
 Returns the power of a number.
 Example: Select POWER (2, 3) FROM DUAL;

Character (String) Functions in SQL

1. **UPPER()** - Convert to Uppercase
 Converts all characters to a string to capital letters.
 Example: Select UPPER ('bca student') FROM DUAL;

2. **LOWER()** - Convert to Lowercase
 Converts all characters to a string to small letters.
 Example: Select LOWER ('Bca Student') FROM DUAL;

3. **INITCAP()** - First Letter Capital
 Converts the first letter of each word to uppercase.
 Example: Select INITCAP ('bca database student') FROM DUAL;

4. **LENGTH()** - Find length of string
 Returns the number of characters in a string.
 Example: Select LENGTH ('bca database student') FROM DUAL;

5. **SUBSTR()** - Extract part of string
 Returns a substring from a string.
 Example: Select SUBSTR ('bca database', 1, 4) FROM DUAL;

6. **INSTR()** - Find position of character in string
 Returns the position of the first occurrence of a character in a string.
 Example: Select INSTR ('bca database', 'a') FROM DUAL;

7. **REPLACE()** - Replace character in string
 Replaces a character in a string with another character.
 Example: Select REPLACE ('bca database', 'a', 'x') FROM DUAL;

8. **CONCAT()** - Concatenate strings
 Concatenates two or more strings.
 Example: Select CONCAT ('bca', ' database') FROM DUAL;

9. **SPLIT()** - Split string into array
 Splits a string into an array of substrings.
 Example: Select SPLIT ('bca database') FROM DUAL;

INTEGRITY CONSTRAINTS

What is constraint? ^{5M}

Constraints

- Constraints is a set of rules which ensure that data is stored under certain condition.
- Constraints are conditions applied to table to ensure valid and consistent data in the database.

Type

1. Primary key constraints (keys) ^{5M} explain different relationship.
2. Domain constraints ^{5M} explain different constraints.
3. Integrity constraints ^{5M} explain different constraints.
4. Foreign key constraints ^{5M} explain different constraints.

Key (constraints keys) ^{5M} a word in a table, type is the key. Important constraint on the entities of an entity type is the key. - used to connect table uniquely. constraint on attributes. - avoid duplicate data.

Use of keys ^{5M} maintain data integrity. Mainly for unique identification. Identity of the data and ensure integrity of the data.

Identify relationship between the table. Avoid duplicate data.

Types of keys

1. Super key
2. Candidate key
3. Primary key & alternative key
4. Unique key
5. Foreign key
6. Composite key & compound key

DATE Function in SQL

1. SYSDATE - Current date and time

- Returns the current system date and time.

Syntax

select sysdate from dual;

2. ADD MONTHS() - Add Months to Date

- Add a specified number of months to a date.
- Add 3 months to the current date.

Syntax

select add_months(sysdate, 3) from dual;

3. MONTHS - BETWEEN() - Difference b/w Months

- Returns the number of months between two dates.

Syntax

select months_between(sysdate, '16-March-2026') from dual;

4. NEXT_DAY() - Find Next Day - next day returns

- Returns the next specified weekdays next coming date.

Syntax

select next_day(sysdate, 'MONDAY') from dual;

5. LAST_DAY() - Last Day of Month

- Returns the last date of the month.

Syntax

select last_day(sysdate) from dual;

Returning ind. • Format is DD-MM-YYYY.
 01P = 01P = 30-APR-2026

- ① super key
any set of attributes that uniquely identifies a row of data in a table.
- The super key attribute values can be null.
- All the super keys joined together to bring the candidate keys.

Reg no	Name	Place	Blood group	DOB	Adhar No	Mob No
a	1	Amrutha	A+	3-1-2006	1764 76548100	97536281180
b	2	Anil	O+	5-7-2006	6870 1358 9811	8056781813
c	3	Ratna	A+	3-1-2006	1481 6799 1426	9900754316
d	4	Sudip	AB+	30-6-2006	1680 7886 3412	6363899610
e	5	Ashray	B+	1-1-2006	6882 9933 6519	7935348813

Set of Super key

✓ Reg no

(Adhar no)

(Reg no, name)

(Reg no, Blood group, Address)

(Mob no, DOB)

(Reg no, Adhar no, Address)

(Reg no, Mob no)

(Name, Blood group, Adhar no, Mob no, DOB)

(Reg no, name, Blood group)

(Adhar no, Blood group, Mob no)

(Reg no, Blood group, name) & so on...

- ② Candidate key
It is an attribute that uniquely identifies a row in a table.

① Unique identification

② No redundancy

It is a subset of the part of super key.

All candidate keys are super keys.

Its values cannot be null.

Ex: Reg no, column, Roll no, Adhar no, Mob no.

Roll no	Reg no	Name	Place	Blood group	DOB	Mob No	Adhar No
1	a	AAA	Nargal	A+	16-03-2000	9913306583	18706100
2	b	ABC	Mysor	O+	19/11/2000	8365135463	12345678
3	c	ABG	Belagavi	AB+	01-06-2000	9900754316	7676355048
4	d	ACD	Hubbali	B+	30-06-2000	6566735819	3162236688
5	e	ABG	Mysor	B+	16-03-2000	7999876891	28268791345

candidate key

3M defn

Primary key

(500)

Primary key is the candidate key which uniquely identifies row in table.

Every relation must contain a primary key.

One candidate key is chosen as primary key.

No null value

Primary key cannot contain duplicate values.

A relation can contain only one primary key.

any one is candidate key we have to choose primary

1M

The super key is the super set, the candidate key is the subset and the primary key is the sub-subset of the super key.

Regno / Roll No - Primary key

Reg No	Name	Address	Blood group	DOB	Adhar No	Mob No
1	Ashray	Bhaktal	O+	3-1-2003	01786547825	180786
2	Adarsh	Bhaktal	A+	14-11-2003	19870387627	730558
3	Adarsh	Bhaktal	A+	14-11-2003	19873777987	829658
4	Ravi	Kumari	B+	30-6-2004	17654673519	908866
5	Baskar	Hannan	C+	4-12-2004	68887835489	900808

primary key

Division key

* 1 column no issue

* 2 cols have no issue with it
choose primary key as foreign key

Create table students-details (regno as primary

key, name varchar(10), address varchar(10),
bloodgroup varchar(5), DOB date, Adhar no int,
mobno int);

primary key no issue with it
foreign key no issue with it

Alternative key (Secondary key)

An alternate key is a secondary candidate key that is capable of identifying a row uniquely

Wife a note on Foreign key

Foreign key

It is an attribute in a table which is used to define its relationship with another table.

It means the columns of one table points to the primary key attribute of the other table.

It is used for creating and maintaining the relationship between the tables

Reg No.	Name	Address	Blood group	DOB	Adhar No	Mob No.
1	Ashray	Bhaktal	O+	3-1-2003	01786547825	180786
2	Adarsh	Bhaktal	A+	14-11-2003	19870387627	730558
3	Adarsh	Bhaktal	A+	14-11-2003	19873777987	829658
4	Ravi	Kumari	B+	30-6-2004	17654673519	908866
5	Baskar	Hannan	A+	4-12-2004	68887835489	900808

Foreign key (because it is common in both tables)

Reg No	Total class attended
1	9
2	8
3	8
4	8
5	1

Create table attendance-info (regno reference student details, total-class-attended int);

Foreign key is a reference to the primary key of another table

primary key and foreign key

- Unique key constraint is used to ensure that all values in a column are different.
- It prevents duplicate values in the specified columns.
- Unique constraint allows null values in a table have the same value in a the specified column.

Syntax

create table table-name (column-name datatype unique);

create table students (regno int unique);

alter table students add constraint unique_email unique (email);

regno	name	address	bloodgroup	DOB	adhar	mobno
1	Adarsh					
2	Akshay					
3	Aryan					
4	Adarsh					
5	Akshay					
6	Aryan					

regno { sup key
adhar { sup key
mobno { sup key

regno { only 1
adhar { only 1
mobno { only 1

Difference between UNIQUE and PRIMARY KEY

UNIQUE	PRIMARY KEY
Not allowed	Not allowed
Allowed	Not allowed
Multiple	only one.

Composite key, OR, AND

- it is a primary key made of more than one attribute called by composite key.
- Instead of one column uniquely identifying a row, combination of columns do.
- Each column may have duplicate but together must be unique.

Name	Place	Course	Course id
Adarsh	Bangalore	BFA	BFA11
Akshay	Mysore	BBA	BBA11
Aryan	Bharuach	BBA	BBA11
Adarsh	Bangalore	BFA	BFA11

create table course-info (name, place, year, course varchar(10), place varchar(10), course varchar(10), primary key (name, course-id));

to combine & primary key

Compound key having attributes which is

1. Composite key having attributes which is

2. Domain constraints

Q1/11/17

It helps to enter the data into the table according to the particular datatype.

Ensures what kind of attributes can be stored in a table.

Type

1. Not Null

2. Check

1. Not Null

Not null is used which restricts a column not to accept null value.

2. Check

char(5);

2. Check

It works as a filter that limits the domain some range.

It ensures that the data is entered into table after satisfying certain conditions

Ex: create table emp (empid int not null, name char(5), salary char(5) check (salary > 5000));

Valid condition

create table student (age and check (age >= 18));

3. Integrity constraints

It ensures that the data insertion, updating and other processes have to be performed in such a way that data integrity is not affected

Type

1. Entity integrity constraints (based on primary key)

2. Referential integrity constraints (based on foreign key)

Entity integrity constraints no null value

Primary key value can't be null

Ex

S.No	Name	Register	Age
1	Adarsh	1st	19
2	Rushal	4th	20
3	Akash	3rd	20
	Vishal	7th	21

Describe the concept of integrity constraints

1. Entity integrity constraints (based on primary key)

2. Referential integrity constraints (based on foreign key)

Table 1 and 2, if a foreign key is related to as a primary key of another table then the contents of the foreign key of table 1 must be null.

Example

S.No	Name	Serial code	Code	City
1	Adarsh	1st	4178	Panna
2	Rushal	4th	8451	Bethi
3	Akash	5th	9654	Mumbai
4	Vishal	7th	1858	Pune
5	Sankalp	5th	4178	

What is database integrity? Discuss different types of integrity constraints.

Advantages of Integrity Constraints

1. Ensure Data Accuracy
 - Prevents spurious data entry
 - eg: age 20
2. Notations Data Consistency
 - some data follows same rules across table
 - eg: - No duplicate IDs
 - standard formats
3. Enforces Relationships
 - connects tables properly
 - ensured valid connections b/w tables
 - eg: - Foreign key must exist in parent table.
 - Parent structure: parent table
 - Child structure: child table depends on parent
4. Prevents Duplicate Data
 - Unique
 - Primary key
 - Unique key
 - ensures no repeated records.
5. Improved Data Security
 - Restricts wrong or unauthorized values
 - eg: - Not NULL constraint.
 - Protects important information
 - Prevents data entry

10M

Views (Virtual table) in SQL

- A view in SQL technology is a single table virtual table that is derived from other table.
- Views are just set of stored query.
- These other tables can be base tables or stored data previously defined views.
- A view does not necessarily exist in physical join.
- In constraint to base tables, whose tuple

Views are the various portions of tables. They explain concepts of views with suitable example. are always physically stored in the database.

Uses of using View

- To represent different views of the same data
- Provide the data independence
- To make complex query easy using
- To restrict the data access

Creating Views

Syntax CREATE VIEW view-name AS

SELECT column1, column2, ... FROM table-name

WHERE condition;

Deleting Views

Syntax DROP VIEW view-name;

OR DROP VIEW student-view;

Updating a View

- A view can be updated under certain condition
- The SELECT clause may not contain the keyword DISTINCT, SET operation, ORDER-BY and GROUP BY clause, NOT NULL.
- The SQL UPDATE VIEW command can be used to modify the data of a view
- All views are not updatable, so UPDATE command is not applicable to all views.

possible only if simple view

cannot update complex views

changes affect original table

view (viewing)

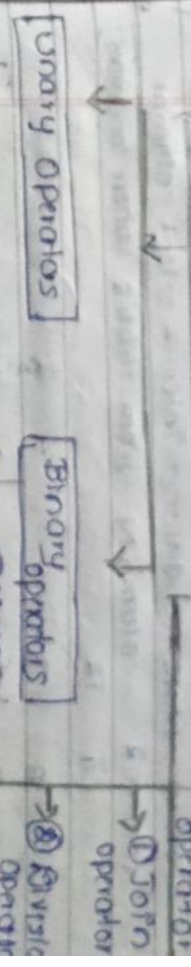
update statements - view SET age=21 where

name = 'Ramu';

Relational Data Model and Relational Algebra

Relational Algebra Operators

Basic operators

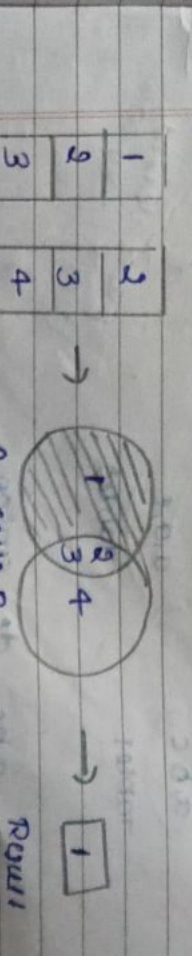


- Unary Operators**
- ① Projection (selection) operator (π)
 - ② selection operator (σ)
 - ③ Rename operator (ρ)
- Binary Operators**
- ① Union operator ($\cup$)
 - ② Cross product operator ($\times$)
 - ③ Minus / set difference operator ($-$)

Set Theoretical operations on Relations (Set Operations)

① Minus (set difference) operator: $A - B$ results in records from the first dataset and then remove from the second dataset.

Syntax: select column... column N From table where conditions minus select column 1... column M from table where conditions;



A - B $\Rightarrow$ returns rows present in first table not in second table.

Syntax for Updating View
 update view-name set column 1 = value1, column 2 = value2... where condition;

Assertions
 Assertions are global constraints applied to the entire database and just one table.
 An assertion is a condition that must always be true for the database.

Example: salary of all employees should not exceed 1,000,000
 create ASSERTION salary-limit check (select sum(salary) from employee) <= 1000000

Note
 They are functional algebra operation with set theory

They are functional algebra operation with set theory

① They are functional algebra operation with set theory

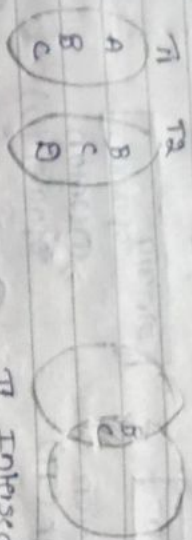
Set Theory
 ① Union
 ② Intersection
 ③ set difference
 ④ cartesian product
 ⑤ set

Q1 What is Union compatibility condition? explain set theory operations in algebra

Two tables can be used in union only if they have the same structure & same type of column.

② Intersection
It returns common or matching records or more select queries.

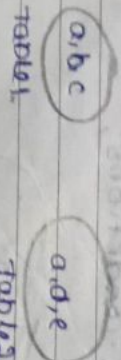
Syntax select column1, column2... column N from table1 where conditions intersect select column1, column2... column N from table2 where conditions;



π Intersect $\cap$

③ Union
Union operator is used to combine the result sets of two or more Oracle select statements. It combines the both select statements and removes duplicate rows but then each select statement within the union operator must have same data type & same number of fields.

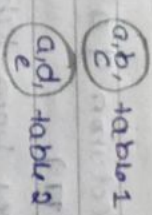
Syntax select column1, column2... column N from table1 where conditions Union select column1, column2... column N from table2 where conditions;



π Union $\cup$

a, b, c, d, e union

④ Union All
The union all operator is used to combine the result sets of 2 or more select statements. Union operator removes duplicate rows while union all operator does not remove duplicate rows.



π Union All $\cup$

⑤ Cross (Cartesian) Product:

It takes two relation as input and gives one relation the cross product as the output. It is denoted by $\times$.

Syntax select table1, column1, table2, column2... from table1, table2, column1, table3, column3... join table3 cross join table4;

Cross product is an operation that combines every row of the first table with every row of the 2nd table.

symbol $\times$

Unary Relational Operations

Example: select & project operation

- ① **Select**
Select operations used to retrieve entire rows from the table.
It helps to display all the records of table.
It is denoted by sigma (σ)

Syntax: select column-names from table-name;

- ② **Projection operation**

- It displays the specific column of a table.
- It is denoted by pi (π)

Syntax: select column-names from table-name;

- ③ **Rename**

- Assigning new name to relation.
- Denoted by R

(i) **Rename column**

Syntax: alter table table-name RENAME column old-name TO new-name;

(ii) **Rename table**

Syntax: Rename table-name to new-table-name;

Binary Relational Operations

Example: Join operation - Join operation

- ① **JOIN clause**

- The join clause takes records from 2 or more tables in a database and combines it together.
- If you want to access more than one table through a select statement
- The joining of 2 or more tables is based on common field b/w them

Types of Join

- ① Inner join
- ② left outer join / left join
- ③ right outer join / right join
- ④ Full outer join / Full join

- ① **Inner join**

The inner join keyword select all rows from both the tables & return records that have matching values in both tables.

Syntax: select table1.column1, table2.column2... from table1 INNER JOIN table2 ON

table1.column-field = table2.column-field;



A B
A B
C C

Example: B B
C C

Ex

Select std. for student- marks from sid inner join student on std.m = student.marks;

M1 MARIOS
Cg CG

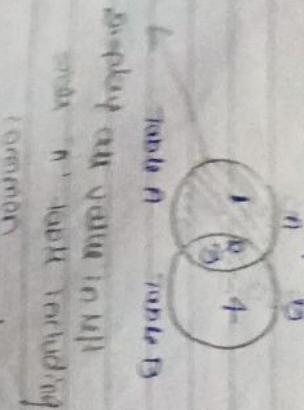
(Both the common in 2 table & they are shown)

② Outer Join Left Join

- Left Outer Join Left Join
- The left join returns all the values from the left table and it also includes matching values from right table
- If there are no matching join values it returns NULL

Syntax

select table1.column1, table2.column2, ...
from table1 left join table2 on
table1.column-field = table2.column-field;



A	B
1	2
2	3
3	4



③ Right Join

Return all records from the right table and matched records from the left table.

Syntax

select table1.column1, table2.column2, ...
from table1 right join table2 on table1
column-field = table2.column-field;



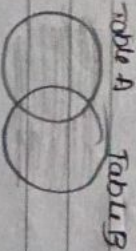
eg:- select stud.mig, student.marks from stud
right join student on stud.mig = student.marks

④ FULL JOIN / FULL OUTER JOIN

- Returns all records even there is a match on either left or right table.
- If the rows from both the tables the rows for which there is no matching, the result set will contain NULL values

Syntax

select * from table1 FULL OUTER JOIN table2
on table1.column_name = table2.column_name;



select * from

Table A

Table B

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

Explain various constraint violations in update operation of relation

SM Constraint violations

- ① Domain constraint violation
When data is invalid for that column.
- ② Primary key violation
When we try to insert duplicate value OR NULL in primary key
- ③ Entity integrity violation
Primary key cannot be null.
- ④ Referential integrity violation
Foreign key must match parent table
- ⑤ NOT NULL violation
Trying to insert NULL in required column
- ⑥ UNIQUE constraint violation
Duplicate values in UNIQUE column

(All other remaining topics are covered in middle of our units)

INTRODUCTION DATABASE ARCHITECTURE

Photo today comes in different forms including numbers, text, audio, image and video.

Photo DBMS - 2011

- A Database is a shared collection of related data which is used to support the activities of a particular organization

Ex: contact details in our phone, software like Microsoft Access, Google drive, MS Excel

What is DBMS?

A database management system (DBMS) is a computerized system that enables users to create and maintain database

The DBMS is a general purpose software system that facilitates the processes of defining, constructing, manipulating and sharing database among various users and applications

Difference b/w DBMS and RDBMS

DBMS VS RDBMS	RDBMS
1. DBMS store data as file	4. RDBMS store data in tabular form
2. Data redundancy (duplicate) is common in DBMS	5. Key and Index do not allow data redundancy
3. Data fetching is slow for large amount of data	6. Data fetching is fast because of the relational approach
4. In DBMS quite complex to alter data	7. In RDBMS it is very easy to alter data
5. DBMS support single user	8. RDBMS supports multiple users
Examples: XML, Microsoft Access etc	Examples: MySQL, SQL server etc

Advantage / Application of DBMS (Introduction of Database Approach)

1. Controlling the redundancy (duplicate)
2. Redundancy means storing the same data multiple times
3. This leads to duplication of effort
4. Second, storage space is wasted when the same data is stored repeatedly, and this problem may be serious for large databases.
5. Third, it is that updating the same data may become inconsistent. This may happen because an update is applied to some of the files but not on others.
6. Restricting unauthorized access
7. When multiple users share a large database, it is likely that most users will not be authorized to access all information in the database.
8. For example financial data such as security and bonus is often considered confidential
9. A DBMS should provide a security and authorization subsystem, which the user uses to grant access and to specify access restrictions
10. The DBMS enforces these restrictions automatically
11. Providing continuous storage
12. Database can be used to provide persistent storage for program objects and data structure
13. The persistent storage of program objects and data structure is an important function of database systems
14. The objects in database structure which can be stored permanently and can be used or accessed

explain actors on the scene - SM

actors on scene classification type

1. Database Administrators
2. Database Designers
3. end-users / Noe programmers
4. system Analysts and Application Programmers (Software engineers)

→ 1. Database Administrators

- Administering database (primary resource) & DBMS and its related software (secondary resource) in the responsibility of the database administrator (DBA).
- DBA is responsible for authorizing access to the database.
- DBA is responsible for authorizing access to the database.

- DBA is responsible for authorizing access to the database.
- DBA task is to solve the system problem.

→ 2. Database Designers

- Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data.
- Communicate with our prospective database users typically interact with potential group.

→ 3. end users

- end users are the people whose jobs require access to the database for querying, updating and generating reports.
- Categories of end users are:
 1. Casual end users: who use the database infrequently.
 2. Naive or Parametric end user: who use the database for specific tasks.
 3. Sophisticated end user: who use the database for complex tasks.
 4. Standalone users: who use the database for their own purposes.

4. Providing storage structure for efficient query processing. The database is typically stored on disk. The DBMS must provide specialized data structures to speed up disk search for the desired records.

• The query processing and optimization module of the DBMS is responsible for choosing an efficient query execution plan for each query.

5. Providing backup and recovery

- DBMS provide facilities for recovering from hardware / software failure.
- The backup and recovery subsystem of the DBMS is responsible for recovery.

• For example, if the computer system fails in the middle of a complex update program, the recovery subsystem is responsible for making sure that the data database is restored to the state it was in before the program started executing.

Explain various users of DBMS → DBMS

Database User (People associated with Database system)

eg:- The person who works on or uses the database is called as Database users, or users.

Two main classification

1. Actors on scene.
2. workers behind the scene.

The people whose jobs involve the day-to-day use of large database are called as actors on scene.

Explain different types of database users

Example: In a bank, the person who works on the computer system is called as a database user.

Categories/Type of End Users

- ① Casual end users
 - Casual end users occasionally access the database for different reason
 - They use sophisticated database query interface to specify their request
 - ② Parametric / Native end user
 - Jobs to receive around constantly querying updating / using the database
 - Small portion of the database
 - ex:- Bank withdrawal, ticket booking, etc.
 - ③ Sophisticated end user
 - Scientist, business person, analyst use data for standard analysis.
 - ④ Standalone users
 - Mainly person who maintain personal database
 - Ready-made program package that provide easy-to-use menu based or graphic based format
 - ex:- Personal data, address book, etc.
- ### A. System Analysts and Application Programmers
- (Software engineers) design software systems
- system analysts is analyst how user software, hardware and the wider IT system fit the business needs of their employer or of a client
 - They work requirements for new systems and may also help implementing them and monitor their effectiveness
 - Typical responsibility of the job include: examining current system.

Application programmers

Application programmers implement given specification or programs; then they test, debug, document or maintain these

Explain in brief various actors behind the scene

WORKERS BEHIND THE SCENE

The people whose jobs is to associate with design, development & operation of DBMS, we call them workers behind the scene.

① DBMS System Designer & Implementers

- Main job is to implement DBMS modules and package.
- DBMS software consists of many modules for implementing the catalog, query language processing, interface processing, accessing and bulk loading data, controlling concurrency, and handling data recovery and security
- System interface with OS (compilers)

② Tool App Developers

- Implements the different packages and facilities to design.
- Tools generally include package for DBMS design, monitoring, test data
- Independent developer develops these tools.

③ Operators & Maintenance Personnel

- DBA personnel who are responsible for running, maintenance of the hardware and software environment for DBMS
- These people helps to end user

Characteristics of the Database Approach (DBMS)

1. Self-Describing Nature of a Database System / Database system having the definition and description of database structure and constraints. Database software equally well with all different self-manage. DBMS extract all the required structure of database.
2. Isolation between Programs and Data, and data abstraction. The database approach, programs do not require change in most cases and the data structure is stored in the system. We call this property 'program-data independence'.
 - user can define the operation
 - user can interface the name and the data types of its arguments (or parameters)
 - user can implement the operation through their
 - user can invoke these operations through their names, arguments, regardless of how operations are implemented - PDI.
3. There is operation called as 'Data Abstraction' (program-data independence + P.O.I.)
4. Support of Multiple views of the Data.
 - Each user having different perspective or views
 - Example: user may be interested only in accessing and printing the transcript of each database.

5. Data Storing and retrieving.
 - Data redundancy
 - Data security
 - Data integrity
 - Data sharing
6. Sharing of data and Multi-user Transaction.

7. Processing.
 - DBMS support multiple to access
 - These possible only when multiple application integrated and maintained in a single DB.
 - Having concurrency control.
 - Having time sharing with advantage of DBMS.
8. Implications (Purpose) of Using the Database Approach.
 - Potential for enforcing standards
 - The database approach permits the DBA to define and enforce standards among database users in a large organization.
 - This facilitates communication and cooperation among various departments, projects and users within the organization.
 - Standards can be defined for names and formats of data elements, display formats, report structure, terminology & many more.

9. Reduced Application Development Time.
 - Developing a new application - such as the retrieval of certain data from the database for printing a new report - takes very little time.
 - Development time using a DBMS is estimated to be one-sixth to one-fourth of that for a traditional file system.
10. Flexibility.
 - It may be necessary to change the structure of database as requirements change.
 - For ex- a new user group may emerge that needs information not currently in the database.
 - Modern DBMSs allow certain type of evolutionary change to the structure of the database.

2

④ Availability of Up-to-Date Information

- As soon as one user's update is applied to the database all other users can immediately see this update.
- Helps in transaction-processing applications concurrency control.

⑤ Economy of Scale

- The DBMS approach reducing the amount of wasteful overlap between activities of data-processing personnel in different projects or departments.
- Reduces overall costs of operation and management.

IMP. SN Explain when not to use a DBMS

① The overhead costs of using a DBMS

- High initial investment in hardware, software and training.
- Overhead for providing security, concurrency control, recovery and integrity functions.

② More desirable to develop customized database applications

- I/O & file apps are simple, user-defined database applications that are not expected to change at all.
- Embedded system with limited storage capacity, where a general purpose DBMS would not fit.

③ Certain industry and applications have decided not to use general-purpose DBMS's

for ex:- CAD tools, proprietary file and data management software.

Explain DBMS software - SN

Classification of DBMS -> SN

4. Communication and switching systems designed by company

for ex:- AT & T were early manipulations of database software, GIS implementations often implement their own data organization scheme for efficiently implementing functions.

DATA MODELS / STRUCTURE of DBMS

A collection of concepts that can be used to describe the structure of a database.

- Provides the data abstraction tool.
- Most data models also include a set of basic operations.
- Allows the database designer to specify a set of user-defined operations.

Explain what's data models, explain category

Categories of Data Models Types

- ① High-level (conceptual) Data Models
- ② Low-level (physical) Data Models
- ③ Representational (Record Based) Data Model.
- ④ Object-oriented Data Models
- ⑤ Self-describing data models
- ⑥ Object-Relational Models

explain.

data model, schema, instance -> not

Database Schema

Schema

In a data model, it is responsible to distinguish between the description of the database and the database itself. The description of a database is called the database schema. It is a logical representation of the database.

Schema diagram displays only some aspects of a schema, such as the names of record types and data items and some type of constraints.

Student
Roll No.
Name
Student number
Class
Major

Course
Course name
Course number
Credit hours
Department

Instances (Snapshot / database state)

The data in the database at a particular moment in time is called a database state / snapshot. It is also called the current set of occurrences or instances in the database.

Schema

Schema architecture

external & internal mapping

only the user's view

SQL

actual data stored in the database at a particular moment of time

Data abstraction: hides the complex details from the user and only see required parts.

hides the complex details from the user and only see required parts.

not external mapping

from the user's view

3NF

What is data independence? Explain three-tier schema architecture. — SM.

Three Schema Architecture (3SBA)

End users

external level

External view

External view

external mapping

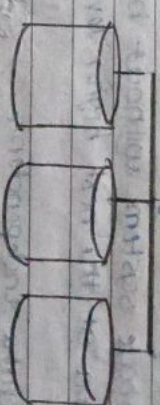
conceptual level

Conceptual schema

conceptual mapping

internal level

Internal schema



stored database

1. The internal level: has an internal schema which describes the physical storage structure of the database.

2. The internal schema: uses a physical data model and describes the complete details of data storage and access paths in the database.

3. The conceptual level: has a conceptual schema which describes the structure of the whole database for a community of users.

The conceptual schema hides the details of physical storage structure and conceals

on describing entities, data type, relationships users operations and constraints.

- usually a representational data model is used to describe the conceptual scheme when a database system is implemented.

- The external view level.
includes a number of external schemas ~~for~~ or user views

- each external schema describes the part of the database that a particular user group is entitled to and hides the rest of the database from that user group.

- Each external schema is typically implemented using a representational data model.

Data Independence
The capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

Types

- ① Logical Data Independence
- ② Physical Data Independence

- The capacity to change the conceptual schema without having to change external schemas

- Change the conceptual schema to expand the database to change constraints, or to reorganize the database.

- We may change the conceptual schema to expand database (by adding a record) or to reduce the database (by removing a record)

2. Physical Data Independence

- The capacity to change the internal schema without having to change the conceptual schema
- The external schemas need not be changed as well

- Example: creating new view
- The exact location of data on disk, and hardware details of storage encoding, placement, compression, splitting, merging of records, and so on are hidden from the user.

Classification of Database management systems

1. Data Model

- The object data model was implemented in some commercial systems but has not had widespread use
- Mainly relational data model, object data model widespread use
- Many older applications still run on database systems based on the hierarchical and network data models.

2. No. of users supported by the system

- Single user - system only one user at a time
- Multi-user system - support concurrent multiple user

3. Number of sites over which database is distributed

- A centralized DBMS can support multiple users, but the DBMS and the database reside totally at a single computer site.
- Distributed DBMS - connected by network and distributed on many sites.

the data is often replicated on multiple s/hg. so that failure of a s/hg will not make some data unavailable

④ Cost

- Many RDBMS products are available as free for 30 days
- Additional cost for new feature
- Installation and maintenance cost of large database systems are usually

database interface

④ DBMS Interface

- Menu-based interface for web clients or browser (Menu based interface)

- Interface present the user with lists of menus that lead the user through the formulation of a request
- Menus do away with the need to memorize the specific commands and syntax of a query language

⑤ Apps for Mobile Device (Mobile App Interface)

- The apps have built-in programmed interface that typically allow users to login using their account name & password
- For ex: banking, reservation among many others provide apps that allow users to access their data
- For ex: banking, reservation among many others provide apps that allow users to access their data

Types

- ① Menu based interface
- ② Form based interface
- ③ Command line interface
- ④ GUI
- ⑤ Natural language
- ⑥ Mobile app interface

⑤ Forms-based interface

- It displays a form to each of user.
- Users can fill out all of the form entries to insert new data or only certain entries in which case the RDBMS will retrieve matching data for the remaining entries
- Forms are usually designed and programmed for native users as interface to canned transactions

⑥ GUI

- GUI typically displays a schema to the user in diagrammatic form
- The user then can specify a query by manipulating the diagram
- In many cases, GUI's describe both menu and forms

⑤ Natural Language Interface

- These interface accept request written in English or some other language and attempt to understand them.
- It having own schema, dictionary, that is used to interpret the request

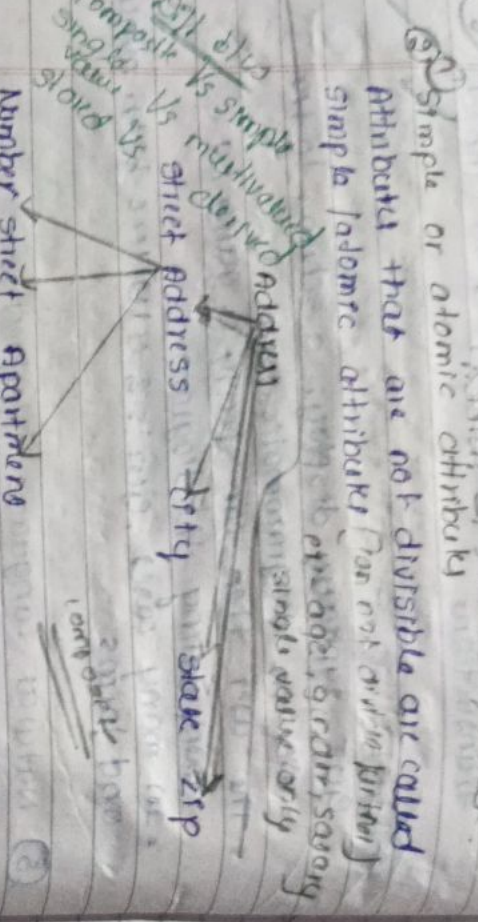
GUI based environment - form

Unit Chapter 3 Data Modeling using the Entity-Relationship (ER) Model Type of Attributes

Attributes are characteristics of an entity. They are used to describe the entity. Attributes can be divided into two types: **Simple Attributes** and **Composite Attributes**.

Simple Attributes are those which cannot be further divided. They are represented by a single oval. Example: Name, Age, Sex.

Composite Attributes are those which can be further divided into smaller attributes. They are represented by a double oval. Example: Address (Street, City, State, Zip).



- ③ Single valued attributes have a single value for a particular entity such attributes are called single-valued. For ex age is a single valued attribute of a person. contains only 1 value for each entity / value.
- ③ Composite attributes can be divided into smaller attributes, which represent more basic attributes can form a meaning, composite attributes can form a hierarchy.

Ex - Role No. Address

③ We represented using double oval
③ can have more than one value

Multivalued Attributes are those which can have multiple values for a particular entity. Each attribute is called multivalued. A multivalued attribute may have lower and upper bounds to constrain the no. of values allowed for each individual entity.

Ex - Different people can have different number of values for the college degree attribute.

Stored and Derived Attributes
Value of data which is fixed called stored. Value for that attribute takes the help of another attribute called derived.

Example
In some cases 2 or more attribute values are related, for ex, the age and Birth-date attribute of a person.

Value of age can be determined from the current (today's) date and the value of that person's Birth-date. The age attribute is hence called a derived attribute and is said to be derivable from the Birth-date attribute which is called, stored attribute.

Derived	It is calculated.	Stored	opposite of derived
Derived from other attributes		Attributes that is directly saved into	
Ex - age (from DOB)		name, salary	
total salary (basic + allowances)			
Derived one			

2. New Attribute

A particular entity may not have an applicable value for an attribute for each situation, a special value called NULL is created.

- Ex: - a college degrees attribute applies only to people with college degrees

Entity

Define Key Attribute - SM

3. Key Attribute is an attribute that is a part of key & helps uniquely identify a tuple (rows) in a relation.

Example :- STUDENT (SID, Name, (course))

SID is a key attribute bcz it uniquely identifies each student.

Ex: - student ID, employee ID, ID number represented using underline

1. Entity Set

An entity, which is a thing or object in the real world with an independent existence, a record (in entity table) may be an object with a physical existence.

- For example, a particular person, car, house or employee.

Weak Entity SM (not)

An entity that depends on another entity called a weak entity.

- The weak entity does not contain any key attribute of its own. The weak entity is represented by a double rectangle.

Entity Type (collection of entities) SM

An entity type is a collection (or set) of entities that have the same attributes.

- Each entity type in the database is described by its name and attributes.

These entities share the same attributes, but each entity has its own value(s) for each attribute.

Entity Set (rows) SM

The collection of all entities of a particular entity type in the database at any point in time is called an entity set or entity collection.

Entity set is a collection of entities of the same entity type.

express the conductivity ratio and $\rightarrow 5 \text{ SM}$
a relationship. Give example.

52

Relationships (cardinally) between cultures

Associations or relationships between an attribute of one entity type and an attribute of another entity type, some relationship exists to another entity type.

ratios are 1:1 (one to one), 1:N

- cardinality $N:M$ (many to many) or $(one\ to\ many)$ and $N:M$ defines the maximum

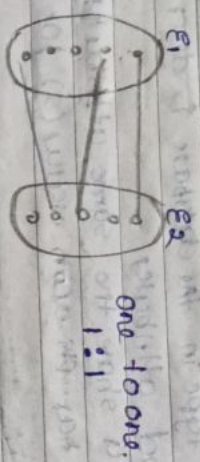
- Cardinality in a relationship instance is the number of relationship instances in which an entity can participate.

Relationship types / cardinality type

4. one-to-one relationship, a new table

A fan have no more than one matching row

an table B, and vice versa. PS created e) both a one to one relationship ^{using} of the related columns are primary keys or have unique constraints)



example:- jare to jare pnrnivis

nut, bolt connect 

Relationship → Relationship shows how entities

DATE	BY	DESCRIPTION	AMOUNT
1/1/20	DR	OPENING BALANCE	1000
1/1/20	CR	SALES	500
1/1/20	DR	COGS	300
1/1/20	CR	EXPENSES	200
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100
1/1/20	DR	RETAINED EARNINGS	1000
1/1/20	CR	DEBT	500
1/1/20	DR	INVENTORY	100
1/1/20	CR	RECEIVABLES	50
1/1/20	DR	PAYABLES	50
1/1/20	CR	EQUITY	1000
1/1/20	DR	DEBT	500
1/1/20	CR	REVENUE	500
1/1/20	DR	EXPENSES	200
1/1/20	CR	INCOME TAX	100

A relationship in ER Model described using 2

① cardinality ratio

② Participation

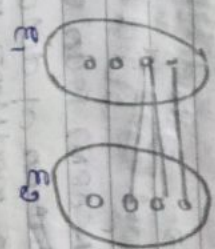
Partnership

159444 CL

One-to-many Relationships General particular

• In this type of relationship, a row in table A can have many matching rows in table B

but a row in table B can
matching row in table 'A')



One to many
1:M

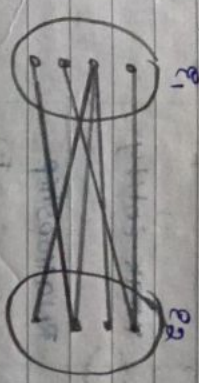
one entity related
to many entities

ex- ① Teacher taking classes to many students
② Doctor treating multiple patients.

② Doctor treating multiple patients

② Many-to-many relationships

Occur when each record in the first table corresponds to one or more records in the second table and each record in the second table corresponds to one or more records in the first table



$N:N$ many to many

et:- a group discussion

• one teacher may teach more than one course and each course may have more than one instructor.

Identifying relationship

- A relationship between two entities in which child entity is identified through parent entity

child entity is dependent on parent entity for its identity

What is ER Diagram? → A diagrammatic way of representing entity & relation ship.

Entity-Relationship model (ERM) is an abstract and conceptual representation of data.

- ER modeling is a DB modeling method, used to produce a type of conceptual schema of a system
- Graphical representation of entities and their relationship to each other is called by

ER Diagram

ER Model

Notations

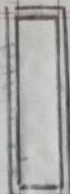
Explaining

Symbol

Entity



Weak entity



Relationship



Identifying relationship



Attribute



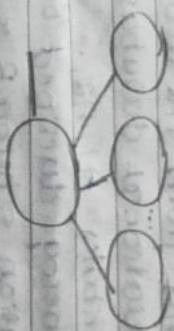
Key attribute



who



Multivalued attribute



Composite attribute

⑨



Derived attribute

Degree of Relationship

Degree of a relationship refers to the number of participating entity type.

- If there are 2 entities involved in relationship then it is called as binary relationship.

Unary

Relational Data Model & Relation Algebra

What is relational data model?

A relational data model is a database model in which data is stored in the form of tables (relations) consisting of rows and columns.

Explain constraints violations in update operation of relations → (first part last part)

List the characteristic of relations →

What is relational database characteristic of relation? relational database constraint types? relation is a table consisting of rows & columns

Functional dependencies and Normalization

for Relational Database

Exp- a function cap-
ble of sets of attributes from the database.
and relational database

- Suppose that our schema has n attributes $A_1, A_2, \dots, A_n$.
- Entire database as being described by a single universal relation schema $R = \{A_1, A_2, \dots, A_n\}$
- A functional dependency, denoted by $\alpha \rightarrow \beta$

Reg No	Name	Place	Nob no
			0608244430

Reg.No	Name	Place	Mob.No
1	Akshay	Bhatkal	868884430
2	Adarsh	Bhatkal	8143655734
3.	Adarsh	Bhatkal	8143655739
4	Anil	Kumta	9898765465
5	Rajkar	Honavane	9988776556

Regno → & Name, Place, Mobno of

This functional dependency specifies that a combination of name, place, mobno is functionally determined by RegNo.

Types of Function

Attributes participating in keys

① Prime attribute - An attribute, which is a part of the candidate key is known as a prime attribute.

⑧ Non-prime attribute - An attribute, which is not a part of the candidate key, is said to be a non prime attribute.

functional dependency $\rightarrow 2NF(84)$

Functional Dependence Types $\rightarrow$ explain the concept of functional dependency & multivalued dependency

Functional Dependency Type, dependency

① Fully Functional Partial $\rightarrow$ 2NF

② Transitive Reproductivity - SMR concept

motivated by

① Fully Functional & pricing : not possible

-nt on another attribute, $q_1 r_1$ is

Functionally dependent on that attribute and not on any of its proper subset

rule ⑧ In few functional dependency, the non-prime attribute is functionally dependent on the candidate key

Fig:-

Round	Coursed	Fees
1	A00000	10,000
1	B	15000
2	A00000	20000
3	A00000	20000
4	A00000	20000
4	A00000	10000

Candidate Key: ROUNO COURSEID (combination)

- Non Prime attributes: Fees depend on candidate

How it works?

from the above table, neither Rollno nor CousinID can uniquely determine the pres

but both Rolando and I agreed together
can identify uniquely. So fees attribute

is fully functionally depends on β and α ,
coupled of

- Rollno (ursetd) → Fees

② Transitive Dependency occurs when an indirect relationship is called Transitive functional dependency it is called Transitive Dependency

• If $X \rightarrow Y$ and $Y \rightarrow Z$ is true, then $X \rightarrow Z$ is a transitive dependency

Roll No	Address	Pin code
1	Bhatkal	1111
2	Honnawara	1112
3	Kanwar	1113
4	Kanwar	1113
5	Bhatkal	1111

• Rollno $\rightarrow$ Address, Address $\rightarrow$ Pincode

Rollno $\rightarrow$ Pincode

• In the above table, Rollno is a transitive dependency of Pincode (Rollno $\rightarrow$ Pincode) because student Address is dependent on Rollno of the student and Pincode is dependent on address

③ Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant
 • If $X \rightarrow Y$ and Y is not a subset of X then it is called Non-trivial functional dependency

When can you say that relation is normal?

Rollno	Address	
1	Bhatkal	$X \rightarrow Y$
2	Honnawara	X is not subset of Y
3	Kanwar	Roll no is not part (subset of)
4	Kanwar	
5	Bhatkal	Address

$X \rightarrow Y$

Here Rollno $\rightarrow$ Address is a non-trivial functional dependency, since the dependent Address is not a subset of determinant Rollno

What is Normalization? RM

Remove Redundancy

Normalization (Normal Form) / NF

Normalization is the process of collectively

organizing data into multiple relational table to minimize data redundancy

Normalization is a method to remove all the anomalies & bring the database to a consistent state. Normalization is a process of organizing the data to reduce redundancy

Normalization rules are divided into following

1. First Normal Form / 1NF (basic)
2. Second Normal Form / 2NF (functional)
3. Third Normal Form (transitive)
4. BCNF (Boyce-Codd Normal Form) / 3.3 (both trivial)

1NF

2NF $\rightarrow$ 1NF rule + 2NF rule

Third normal $\rightarrow$ 2NF + new rule of Third normal

BCNF $\rightarrow$ Third Normal Form + new rule

First Normal Form 1NF

Rule: Only one value in a column containing

• A row in table of data must contain repeating group of data

• Each column must have unique value.

• Each row of data must have a Primary key

Impression is (1NF is not 2NF)

• 1NF proposes a scalable table design that can be expanded easily in order to make the data retrieval process much simpler.

• To deal with multivalued attributes, composite attributes, and their combinations

• 1NF discusses relation within relations

Example: (Not 2NF)

Reg No	Name	Address	MobNo.
1	Akshay	Bhatkal	7827262765
2	Adarsh	Bhatkal	7827262765
3	Adarsh	Bhatkal	7827262765
4	Adarsh	Bhatkal	7827262765
5	Basavar	Honnawara	9008987686

There are 3 main techniques to achieve

First Normal Form 1NF :- (Convert 1NF)

(i) Add column (Technique 1)

1) a maximum number of values are known for the attribute, replace the attribute by 2 atomic attribute

2NF is Normalisation Explain 2NF, 3NF

Reg No	Name	Address	MobNo. 1	MobNo. 2
1	Akshay	Bhatkal	7827262765	7827262765
2	Adarsh	Bhatkal	7827262765	7827262765
3	Adarsh	Bhatkal	7827262765	7827262765
4	Adarsh	Bhatkal	7827262765	7827262765
5	Basavar	Honnawara	9008987686	9008987686

(ii) Add row (Technique 2)

Expand the key so that there will be a separate tuple in the original relation

RegNo	Name	Address	MobNo.
1	Akshay	Bhatkal	7827262765
2	Adarsh	Bhatkal	7827262765
3	Adarsh	Bhatkal	7827262765
4	Adarsh	Bhatkal	7827262765
5	Basavar	Honnawara	9008987686

(iii) Technique 3 add new table.

Remove the attribute that violate and place it in a separate relation (table) along with the primary key

What is normalised 2NF - explain

Table 1

Reg. No	Name	Address
1	Ashay	Bhatkal
2	Adarsh	Bhatkal
3	Adarsh	Bhatkal
4	Anil	Kurda
5	Basavar	Honnavare

Table 2

Reg. No	Roll No
1	7827262765
2	9898767897
3	4365538863
4	9926542868
5	7656567811
6	9826688098
7	9977827711

Out of these 3 techniques / solutions above the last 2s generally considered best because it does not suffer from redundancy (original table) ~~2NF~~ 3NF

- Second Normal Form (2NF) is achieved when every non-prime attribute is fully functionally dependent on the primary key of R.
- Second Normal Form (2NF) is based on the concept of fully functional dependency.
- To achieve 2NF, eliminate the fully functional dependency.

2NF is achieved when every non-prime attribute is fully functionally dependent on the primary key.

Rule: 2NF should meet the requirements of First Normal Form (1NF). All non-key attributes are fully functionally dependent on the primary key.

Roll No	Course ID	Fees
1	A	10000
2	B	15000
3	B	80000
4	B	2000
5	B	2000
6	B	10000

Candidate key: RollNo, CourseID
 Prime Attribute: RollNo, CourseID
 Non Prime Attribute: Fees

- RollNo → Fees
 i.e. Fee is dependent on RollNo, which is proper subset of the candidate key.
- Non-prime attribute Fee is dependent on a proper subset of the candidate key and so the relation is not in 2NF.
- To convert the above relation to 2NF, we need to split the table into 2 tables such as

Course ID	Fees	Roll No	Course ID
A	10000	1	A
B	15000	2	B
B	80000	3	B
B	2000	4	B
B	10000	5	B

Non key attribute - not part of primary key

3) 3NF again 3NF & 3NF is based on

- (3NF) third normal dependency
- the concept of transitive dependency
- $X \rightarrow Y$ and $Y \rightarrow Z$ is transitive dependency
- To achieve 3NF, eliminate the transitive dependency.

- Rule 1) If satisfies 3NF and no nonprime attribute of 3NF transitive dependency on the primary key
- It should not contain transitive dependency.

Rollno	Address	Pincode
1	Bhatkal	581330
2	Honnavara	581334
3	Bhatkal	581330
4	Kurtha	581332
5	Honnavara	581334

Prime attribute - Rollno (part of primary key)
Nonprime attributes $\rightarrow$ Address, Pincode.

- In the above table, the non-prime attribute transitively dependent on the candidate key
- This will violate the rule of third normal form
- To make this table comply with 3NF we have to break the table into 2 tables to remove the transitive dependency

parent table

Rollno	Address
1	Bhatkal
2	Honnavara
3	Bhatkal
4	Kurtha
5	Honnavara

Address	Pincode
Bhatkal	581330
Honnavara	581334
Kurtha	581332

4) BCNF
What is normal/candidate key?
must be candidate key
hand side (LHS) must be split table
in some way
data on attribute must or not

- Boyce-Codd normal form (BCNF) was found to be stricter than 3NF
- A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow Y$ holds in R then X is a candidate key of R

Rule

- X cannot be a non-prime attribute
- BCNF, the table should be in 3NF, and every Functional Dependency of LHS should be super key or candidate key

Rollno	Address
1	Bhatkal
2	Honnavara
3	Kurwar
4	Kurwar
5	Bhatkal

Here, Rollno is prime attribute and address is non prime, since the LHS should be super key or candidate key, so this table satisfies BCNF rule.

Q. What is a primary key? Ans: A primary key is a set of attributes in a relation that uniquely identifies each tuple in the relation.

Q. What is a foreign key? Ans: A foreign key is a set of attributes in a relation that refers to the primary key of another relation.

Q. What is an Anomaly?
 Ans: An anomaly is a problem that occurs in the database due to the duplication of data. It can be of three types: Insertion anomaly, Deletion anomaly, and Update anomaly.

Example

Student ID	Student Name	Course	Faculty (Teacher)
101	Ravi	DBMS	Kumar
102	Sita	DBMS	Kumar

Q. Explain the three types of anomalies.
 Ans: 1. Insertion anomaly: When you cannot insert a new tuple into a relation because it would violate a functional dependency. 2. Deletion anomaly: When deleting a tuple from a relation results in the loss of information. 3. Update anomaly: When updating a tuple in a relation results in inconsistent data.

Q. What is a functional dependency? Ans: A functional dependency is a relationship between two attributes in a relation. If attribute A is functionally dependent on attribute B, then for every value of B, there is exactly one value of A.

Attribute	Value
Course	1
Faculty	2
Student	4
Teacher	2

Q. What is a join operation? Ans: A join operation is a binary operation that combines two relations into a single relation. There are several types of join operations: Natural join, Equijoin, and Theta join.

Q. What is a Relation? Ans: A relation is a table consisting of rows and columns. It is a set of tuples that share a common attribute set.

Q. What are the characteristics of a relation? Ans: 1. Atomicity: Each attribute in a relation must contain atomic values. 2. Uniqueness: No two tuples in a relation can be identical. 3. Order: The order of attributes in a relation is not important.

Q. What is a tuple? Ans: A tuple is a row in a relation. It is a set of values that correspond to the attributes of a relation.

Q. What is a functional dependency? Ans: A functional dependency is a relationship between two attributes in a relation. If attribute A is functionally dependent on attribute B, then for every value of B, there is exactly one value of A.

Q. What is a join operation? Ans: A join operation is a binary operation that combines two relations into a single relation. There are several types of join operations: Natural join, Equijoin, and Theta join.

Join operation → Equi & Natural join - SM
 constraint violation - SM.

- NAME: _____
DATE: / /
- ① What is Relation, characteristic of relation - 10M
 - ② Functional dependency → natural & equijoin - 5M
multivalued & functional dependency
 - ③ View, features & concept of view with ex - 5M/10M
 - ④ Join operation → equi & natural join - 5M
 - ⑤ constraint violation - 5M Same ex. note.
 - ⑥ Anomalies → type insertion, deletion, modification - 5M
 - ⑦ responsibility of DBA/ database designer - 5M
 - ⑧ Schemas, instance
 - ⑧ Explain assertion with suitable examples

Types of data model

Anomaly - Insertion, deletion, modification
Assertion
Constraint violation